

目录

视频生成方法

1. 问题本质：什么叫“视频生成”？

- 1.1 狭义视频生成
- 1.2 广义视频生成

2. 视频生成的六大阵营

2.1 阵营 A：纯生成式视频

典型子类

- A1. Text-to-Video
- A2. Image-to-Video
- A3. Audio-native Video

这一路线的本质

2.2 阵营 B：可控生成式视频

典型子类

- B1. Storyboard / Timeline 控制
- B2. 首尾帧控制
- B3. 多参考图引导
- B4. Video Extension / Continuation
- B5. Prompt Rewriter / Prompt Enhancement

这一路线的本质

2.3 阵营 C：编辑式视频生成

典型子类

- C1. Remix / Recut / Blend / Loop
- C2. 图像/视频局部增强与风格变化
- C3. 基于现有视频的变换

这一路线适合什么

2.4 阵营 D：文档/模板/讲解式视频

典型子类

D1. 文档转讲解视频

D2. 模板视频

D3. 企业培训/说明片自动化

这一路线的本质

2.5 阵营 E：数字人 / Talking Head / Avatar 视频

典型子类

E1. 企业数字人视频

E2. Photo Avatar / Digital Twin

E3. Speaking Portrait

这一路线适合什么

2.6 阵营 F：代码与数据驱动视频

典型子类

F1. Remotion：React 驱动的视频渲染

F2. FFmpeg：滤镜图与底层拼接

F3. MoviePy：Python 非线性编辑

F4. Manim：脚本化动画引擎

这一路线的本质

3. 从“输入形式”再分一遍：视频生成到底有哪些输入法

3.1 纯文本输入

3.2 文本 + 图像

3.3 文本 + 多参考图

3.4 首尾帧 / 锚点帧

3.5 视频输入

3.6 文档 / PPT / 模板输入

3.7 脚本 / 数据 / 代码输入

4. 从“控制粒度”再分一遍：控制力到底差在哪

4.1 语义级控制

4.2 镜头级控制

4.3 时间轴级控制

4.4 结构级控制

4.5 渲染级控制

5. 底层技术路线：真正的“技术流”分类

5.1 Diffusion / Latent Diffusion / Diffusion Transformer

这一类方法的特点

5.2 Latent Compression / 3D VAE / 时空 patch 表示

这一类方法解决什么问题

5.3 Autoregressive / Interactive World Model 路线

这一类路线的意义

5.4 Audio-Visual Co-generation

这意味着什么

5.5 检索/模板/编排型路线

6. 为什么你写很长的 Prompt，仍然控制不住

6.1 因为系统目标不是逐条执行

6.2 因为很多系统会自动改写 Prompt

6.3 因为视频生成是强耦合时空问题

7. Remotion 到底属于哪里

7.1 归类

7.2 方法论地位

7.3 为什么它更适合你

8. 面向生产的最终分类：你该如何选路线

8.1 你要“惊艳感”和创意探索

8.2 你要“文档快速成片”

8.3 你要“强控制工程视频”

8.4 你要“企业级流水线”

9. 对你当前场景的直接判断

9.1 你的场景不适合把 NotebookLM 当导演

9.2 你更适合“AI 策划 + 程序化渲染”

10. 最后的方法论总结

10.1 行业正在从“文生视频”走向“可控视频系统”

10.2 真正完整的视频生成，不止一条路线

10.3 对工程型用户，终局不是更长 Prompt

视频生成方法

1. 问题本质： 什么叫“视频生成”？

1.1 狭义视频生成

狭义上，行业里常说的视频生成，是指模型直接从文本、图像或视频条件生成新视频。

OpenAI 的 Sora 明确支持从文本或图片开始生成视频，并在新产品形态里加入声音；Google 的 Veo 3.1 支持 text-to-video、image-to-video、video extension、首尾帧指定和多参考图引导；Runway Gen-4.5 也明确提供 text-to-video 和 image-to-video，并强调复杂指令执行、镜头编排和提示遵循能力。([OpenAI](#))

1.2 广义视频生成

广义上，任何 **把“内容意图”自动转成视频输出** 的系统，都属于视频生成。

因此，NotebookLM 这种“把文档转成讲解视频”的产品，Synthesia/HeyGen/D-ID 这类“把脚本转成数字人口播视频”的产品，以及 Remotion/FFmpeg 这类“把代码和数据转成视频”的系统，都应纳入视频生成版图。([Google帮助](#))

2. 视频生成的六大阵营

2.1 阵营 A：纯生成式视频

这是最容易被理解成“AI 视频生成”的阵营。输入通常是文本或图像，输出是一段全新合成视频。

典型子类

A1. Text-to-Video

直接从文本描述生成视频。

Sora 官方页面说明可以从 prompt 开始生成视频；Veo 官方文档说明支持 text-to-video；Runway Gen-4.5 也将 text-to-video 列为核心能力。([OpenAI](#))

A2. Image-to-Video

输入一张静态图，生成运动视频。

Runway 的图生视频提示指南明确说：图像负责构图、主体、光线和风格，文本主要描述运动、镜头和时间推进；Veo 3.1 允许图像引导；Sora 也支持从图片开始生成。([Runway](#))

A3. Audio-native Video

视频与声音同步联合生成。

Sora 新产品页和 Sora 2 页面都强调会自动包含音乐、音效与对白；Veo 3.1 文档明确写到“natively generated audio”。([OpenAI](#))

这一路线的本质

它的核心是：

模型负责“想象”画面和运动。

这类系统的优点是灵感强、上手快、画面可能惊艳；缺点是 **不确定性高、复现性差、长视频控制弱、精确执行差**。

这也是你在 NotebookLM/Prompt-only 工作流里感受到“怎么写都不完全听话”的根源之一。Sora、Veo、Runway 的官方文档都在不断补“控制接口”，这本身就说明单纯自然语言还不够稳定。([OpenAI Help Center](#))

2.2 阵营 B：可控生成式视频

这是近一两年最重要的演进方向。

行业不满足于“生成一段漂亮视频”，而是开始提供更强的 **时间控制、镜头控制、约束控制、结构控制**。

典型子类

B1. Storyboard / Timeline 控制

Sora 官方帮助文档提供 Storyboard：可以在不同时间点放入文本、图片或视频卡片，并拖拽它们在时间线中的位置来控制节奏和场景连接。([OpenAI Help Center](#))

B2. 首尾帧控制

Veo 3.1 支持指定 first and last frames 来生成视频，这本质是在时间轴两端给模型打锚点。([Google AI for Developers](#))

B3. 多参考图引导

Veo 3.1 支持最多三张参考图来引导生成内容。

这类控制方式的意义是：你不再只靠语言描述主体，而是直接给模型“视觉约束”。([Google AI for Developers](#))

B4. Video Extension / Continuation

Veo 3.1 支持对已生成视频进行扩展；Sora 的编辑器支持 re-cut、loop、blend、remix。

这说明现在很多视频模型已经不只是“一次性生一段”，而是在向 **可迭代、可编辑、可接续** 发展。([Google AI for Developers](#))

B5. Prompt Rewriter / Prompt Enhancement

Veo 官方明确提供 prompt rewriter，并且在部分模型中默认开启；对于 Veo 3/3.1 甚至不能关闭。重写器会自动补充视频描述、镜头运动、音效与转录信息。([Google Cloud Documentation](#))

这一路线的本质

它的核心不是“让 Prompt 更长”，而是：

给模型更多显式结构化约束。

也就是说，行业已经默认承认：

单段自然语言不足以可靠控制复杂视频。

2.3 阵营 C：编辑式视频生成

这类系统不从零开始，而是对已有图像或视频做生成式编辑。

典型子类

C1. Remix / Recut / Blend / Loop

Sora 提供 Remix、Re-cut、Blend、Loop。

这意味着输入不一定是“文本”，也可以是已有视频或已有生成结果，然后要求模型做局部变化、延长、拼接或风格改写。([OpenAI Help Center](#))

C2. 图像/视频局部增强与风格变化

Pika 的官方产品页和定价页公开列出了 Pikadditions、Pikaswaps、Pikatwists、Pikaffects、Pikascenes 等工具，明显属于“在已有素材上加东西、换东西、扭东西、加效果、改场景”的路线。([Pika](#))

C3. 基于现有视频的变换

Sora 的 Blend、Re-cut，Veo 的 extension，本质都属于“条件视频编辑”。这类方法在生产上比纯从零生成更稳，因为它继承了一部分已有时空结构。([OpenAI Help Center](#))

这一路线适合什么

适合：

- 已有一版视频，要做变体
- 已有图像/IP 设定，要赋予运动
- 已有镜头，但想扩展长度或改变局部内容

不适合：

- 从零搭建强逻辑、强结构、长时序汇报片
-

2.4 阵营 D：文档/模板/讲解式视频

这是你当前实际最相关的阵营。

典型子类

D1. 文档转讲解视频

NotebookLM 的 Video Overview 会从文档中提取图片、图表、引文、数字，生成 **AI narrated slides**，支持 Explainer / Brief 两种格式、语言、视觉风格和 steering prompt。([Google帮助](#))

D2. 模板视频

Synthesia 明确提供模板、场景、Avatar 和 AI assistant 工作流，支持从模板或空白画布开始创建视频。([Synthesia](#))

D3. 企业培训/说明片自动化

D-ID 官方把“把文章、培训材料、企业沟通材料转成视频”作为核心使用场景；Synthesia 也强调 training、sales enablement、marketing 等业务视频。([D-ID](#))

这一路线的本质

它不是在追求“导演级镜头控制”，而是在做：

信息压缩 + 叙事编排 + 低成本成片

所以它的优化目标通常是：

- 快速读懂材料
- 自动组织内容

- 自动给出旁白和结构
- 自动排版成片

而不是：

- 保证每页图像必须全屏
- 每个镜头停留秒数严格可控
- 表格必须只在某个时段放大
- 硬切/无动画/无特效严格执行

所以你拿 NotebookLM 去完成工程汇报导演任务，天然会撞边界。

不是你不会写 Prompt，而是它的系统目标就不是这个。([Google帮助](#))

2.5 阵营 E：数字人 / Talking Head / Avatar 视频

这类视频生成，重点不在“场景世界模拟”，而在“一个虚拟讲述者把内容说出来”。

典型子类

E1. 企业数字人视频

Synthesia 提供 stock avatars、custom avatars、scene-based editor；单个视频可包含最多 150 个场景。([Synthesia](#))

E2. Photo Avatar / Digital Twin

HeyGen v2 API 支持 public avatars、photo avatars、instant avatars / digital twin，并可配合语音系统生成视频。([HeyGen](#))

E3. Speaking Portrait

D-ID 的 Talks API 可以从一张照片和文本直接生成 talking avatar 视频，并返回处理状态和结果视频地址。([D-ID](#))

这一路线适合什么

适合：

- 企业培训
- 内部宣导
- 市场营销
- 教学解释
- 标准化口播

不适合：

- 复杂镜头语言
 - 空间调度复杂的叙事短片
 - 强视觉叙事、强电影语言片段
-

2.6 阵营 F：代码与数据驱动视频

这是 Remotion 所在的阵营，也是你后面真正应该高度重视的阵营。

典型子类

F1. Remotion：React 驱动的视频渲染

Remotion 官方定义非常清楚：

“Create real MP4 videos with React”，可参数化内容、服务端渲染、构建应用。

([Remotion](#))

这不是 AI“猜”视频，而是：

你用代码定义视频。

F2. FFmpeg：滤镜图与底层拼接

FFmpeg 官方文档说明了 filtergraph、overlay、crop、vflip、drawtext、fade 等大量时序与图层能力。

它本质是低层视频处理与渲染流水线引擎。([FFmpeg](#))

F3. MoviePy：Python 非线性编辑

MoviePy 在 PyPI 的官方说明中明确把自己定义为 Python 视频编辑库，支持剪切、拼接、标题插入、合成和自定义特效。([PyPI](#))

F4. Manim：脚本化动画引擎

Manim Community 官方主页将其定义为“Python library for creating mathematical animations”。虽然主要面向数学动画，但它体现的是同一种思想：

动画不是猜出来的，而是编排出来的。([Manim Community](#))

这一路线的本质

这一路线不是“生成”，而是：

程序化构图、程序化排版、程序化时间轴、程序化渲染。

优点是：

- 可控性极高
- 可复现
- 可批量生成
- 可接数据源
- 可做企业级模板化生产

缺点是：

- 前期搭建成本高
- 需要工程能力
- 画面创意感不是自动送你的



3. 从“输入形式”再分一遍：视频生成到底有哪些输入法

这是方法论上非常重要的一层，因为不同系统的“可控性上限”取决于输入接口。

3.1 纯文本输入

典型是 text-to-video。

适合发散创意，不适合强约束片子。Sora、Veo、Runway 都支持。([OpenAI](#))

3.2 文本 + 图像

即 image-to-video。

图像锁定主体与构图，文本控制运动和镜头，控制力明显高于纯文本。Runway 的官方图生视频指南就直接强调：prompt 应主要描述 motion。([Runway](#))

3.3 文本 + 多参考图

Veo 3.1 支持最多三张参考图。

这类输入适合做角色一致性、风格一致性、物体一致性。([Google AI for Developers](#))

3.4 首尾帧 / 锚点帧

Veo 的 first and last frames 就是典型例子。

这类输入的优势是：能把“运动区间”压缩成更清晰的约束问题。([Google AI for Developers](#))

3.5 视频输入

用于 extension、remix、blend、video-to-video。

好处是保留已有时序结构。Sora 和 Veo 都在往这条路补强。([OpenAI Help Center](#))

3.6 文档 / PPT / 模板输入

NotebookLM、Synthesia、D-ID 这类系统更重视结构化内容输入。([Google帮助](#))

3.7 脚本 / 数据 / 代码输入

Remotion、FFmpeg、MoviePy、Manim 都属于这一类。 ([Remotion](#))

4. 从“控制粒度”再分一遍：控制力到底差在哪

4.1 语义级控制

“一个人走在雨夜东京街头。”

这是 prompt 级控制，语义大、约束弱。Sora、Runway、Veo 都支持。([OpenAI](#))

4.2 镜头级控制

“低角度近景，慢推，主体在 2 秒后回头。”

Runway 的提示指南已经强调 camera work 和 temporal progression；Veo 的 prompt guide 和重写器也会显式补镜头描述。([Runway](#))

4.3 时间轴级控制

Sora storyboard 属于这一层。

这时你不再只写一句话，而是定义“什么在什么时候发生”。([OpenAI Help Center](#))

4.4 结构级控制

NotebookLM 的 Explainer / Brief、Synthesia 的 scene/template，属于结构级控制。

它们不是逐帧控制，而是“内容单元控制”。([Google帮助](#))

4.5 渲染级控制

Remotion、FFmpeg、MoviePy 属于渲染级控制。

这里控制的是坐标、时长、图层、转场、透明度、文本、数据绑定。([Remotion](#))

5. 底层技术路线：真正的“技术流”分类

这里讲的是模型与系统机制，而不是产品 UI。

5.1 Diffusion / Latent Diffusion / Diffusion Transformer

这是当前主流前沿视频模型最关键的技术大类之一。

OpenAI 在 Sora 技术报告中明确写到：Sora 是 **text-conditional diffusion model**，使用 **transformer architecture**，在 **compressed latent space** 上对 **spacetime patches** 建模；CogVideoX 论文也明确将自己定义为 large-scale diffusion transformer，并结合 3D VAE 压缩视频。([OpenAI](#))

这一类方法的特点

- 画质高
- 细节强
- 更适合高保真生成
- 推理成本通常较高
- 长视频一致性仍是难点

这也是为什么商业系统会不断补 storyboard、首尾帧、扩展和参考图等外部控制接口。([OpenAI](#))

5.2 Latent Compression / 3D VAE / 时空 patch 表示

Sora 技术报告明确提到先把视频压缩到低维 latent 空间，再切成 spacetime patches；CogVideoX 也强调 3D VAE 对时间和空间同时压缩。([OpenAI](#))

这一类方法解决什么问题

视频太大，直接在像素空间做生成代价过高。
所以主流做法不是直接预测所有像素，而是：

先压缩，再在 latent 时空块上建模，再解码回视频。

这属于当前前沿视频模型的重要共同思路。([OpenAI](#))

5.3 Autoregressive / Interactive World Model 路线

这条路线更强调：

- 未来帧一步步往后滚
- 保持世界状态
- 支持交互、行动条件、长期一致性

OpenAI 在 Sora 报告中把视频模型和“world simulators”联系起来；近期研究也直接把重点转向 interactive video generation 与 world models。([OpenAI](#))

这一类路线的意义

它不只是“生成一段好看视频”，而是想让模型：

持续模拟一个可演化的世界。

这对游戏、机器人、仿真、长视频叙事都更关键。

但这仍是快速演化中的方向，不是你现在做企业汇报视频的首选生产路线。

([OpenAI](#))

5.4 Audio-Visual Co-generation

Sora 2 和 Veo 3.1 都明确支持原生音视频联合生成。([OpenAI](#))

这意味着什么

以前常见 workflow 是：

先生成视频
再单独配音 / 配乐 / 加音效

现在前沿模型开始做：

画面 + 对白 + 环境音 + Foley 联合生成

这对创意短片很强，但对工程汇报片并不一定更优，因为你很多时候更希望 **音频是可控脚本，不是模型自由发挥**。([OpenAI Help Center](#))

5.5 检索/模板/编排型路线

NotebookLM、Synthesia、D-ID 这类系统底层未必把“从零生成世界”作为核心；它们更像：

- 检索材料
- 组织结构
- 套用模板
- 配音成片
- 驱动数字人

这是一条 “内容生产系统”路线，不一定是“世界模拟视频模型”路线。([Google帮助](#))

6. 为什么你写很长的 Prompt，仍然控制不住

6.1 因为系统目标不是逐条执行

NotebookLM 的官方定义就是把 source 变成 narrated slides，并通过 steering prompt 去“focus on specific topics, target a specific audience, provide context”。这类 prompt 更像“偏好引导”，不是“逐镜头硬约束”。([Google帮助](#))

6.2 因为很多系统会自动改写 Prompt

Veo 官方明确存在 prompt rewriter，而且 Veo 3/3.1 不能关。

这意味着你输入的 prompt，不一定就是模型最终消费的 prompt。([Google Cloud Documentation](#))

6.3 因为视频生成是强耦合时空问题

一句 prompt 要同时决定：

- 主体
- 构图
- 风格
- 运动
- 镜头
- 时长
- 场景连接
- 音效/对白

这远比图像生成复杂，所以系统会不断增加结构化控制接口。Sora 的 storyboard、Veo 的首尾帧和扩展、Runway 的 motion/camera prompt guide，都是在补这一短板。([OpenAI Help Center](#))

7. Remotion 到底属于哪里

7.1 归类

Remotion 不属于“纯生成式视频”，而属于：

代码驱动视频生成 / 程序化视频生产

它最接近上面的 **阵营 F**。

Remotion 官方首页直接写的是：用 React 创建真实 MP4 视频，可参数化内容、服务端渲染、构建应用。([Remotion](#))

7.2 方法论地位

如果把视频系统按“谁负责决定结果”来分：

- Prompt 视频：**模型决定很多**
- NotebookLM/Synthesia：**系统模板决定很多**
- Remotion：**你决定很多**

因此，Remotion 更像：

视频领域的前端工程系统，而不是 AI 灵感工具。

7.3 为什么它更适合你

因为你的典型诉求是：

- 画面全屏
- 硬切
- 页间顺序固定
- 某页停留几秒
- 图表只能怎么放
- 配音和页严格同步

这类诉求本质不是“生成”，而是“编排与渲染”。

Remotion、FFmpeg、MoviePy 这类路线天然更匹配。([Remotion](#))

8. 面向生产的最终分类：你该如何选路线

8.1 你要“惊艳感”和创意探索

选：

- Sora
- Runway
- Veo
- Pika

因为它们擅长快速出视觉上新鲜的片段。([OpenAI](#))

8.2 你要“文档快速成片”

选：

- NotebookLM
- Synthesia
- D-ID
- HeyGen

因为它们擅长把脚本、文档、培训材料快速转成可用视频。([Google帮助](#))

8.3 你要“强控制工程视频”

选：

- Remotion
- FFmpeg
- MoviePy
- 必要时结合 Manim

因为你要的是 **确定性输出**。([Remotion](#))

8.4 你要“企业级流水线”

选混合路线：

LLM 产出讲稿 / 镜头表 / JSON 时间线
→ Remotion / FFmpeg 按规则渲染
→ TTS 配音
→ 自动字幕 / 封面 / 批量导出

这条路线最适合做：

- 技术汇报视频
- 日报/周报视频
- 自动化研究播报
- 统一模板的企业内容生产

这个结论并非来自某一家产品宣称，而是由各类系统的控制接口边界共同决定的。NotebookLM、Sora、Veo、Remotion、FFmpeg 的官方能力集合，已经足够说明这一点。([Google帮助](#))

9. 对你当前场景的直接判断

9.1 你的场景不适合把 NotebookLM 当导演

NotebookLM 适合：

- 从资料中抽讲解逻辑
- 产出草稿旁白
- 给出章节结构
- 快速做一版 explain video

不适合：

- 严格执行逐页导演约束
- 精确控制时长、镜头、硬切、缩放、版式
- 做工程汇报风格的高可控成片

这个结论和它的官方产品定义一致。([Google帮助](#))

9.2 你更适合“AI 策划 + 程序化渲染”

你的合理工作流应是：

资料 / PPT / Markdown
→ LLM 生成讲稿
→ LLM 生成逐页镜头表
→ LLM 生成 Remotion 参数 / JSON
→ Remotion 渲染
→ FFmpeg 做压制 / 拼接 / 字幕

这样，AI 负责认知劳动，程序负责执行劳动。

这才是工程化的闭环。([Remotion](#))

10. 最后的方法论总结

10.1 行业正在从“文生视频”走向“可控视频系统”

Sora 的 storyboard、Veo 的首尾帧/多图参考/扩展/prompt rewriter、Runway 对 motion/camera prompt 的强化，都说明行业已经进入 **控制增强期**。([OpenAI Help Center](#))

10.2 真正完整的视频生成，不止一条路线

至少应分成：

- 纯生成
- 可控生成
- 编辑生成
- 文档/模板成片
- 数字人口播
- 程序化渲染

10.3 对工程型用户，终局不是更长 Prompt

而是：

更好的中间表示。

这个中间表示可以是：

- Storyboard
- Scene list
- JSON timeline
- Slide plan
- Avatar script
- React components
- FFmpeg filtergraph

只有当“意图”变成结构化中间层，系统才会变得可控。

这也是为什么 Remotion、FFmpeg 这类路线会在真正的生产中越来越重要。

([OpenAI Help Center](#))